

Finding the three suppliers who cost Arora Traders ₹10.46 Cr — and putting the proof on paper

4,800 purchase orders, 380 customer returns and 34 suppliers, 2021-01-01 to 2024-01-01. Every figure in this document is generated by the same pipeline run as the web app and the briefs.

SHORT DELIVERY + RETURNS ₹15.99 Cr All suppliers, the problem statement's formula	BOTTOM THREE, TOTAL ₹10.46 Cr 54% of all losses on 9% of spend	UNTRACED RETURNS 5.3% value misallocated (vs 48% for the stated rule)	BLIND CHECK 3 / 3 bottom three found without the answer key
--	---	--	--

1 · What was asked, and where each answer lives

DELIVERABLE	WHERE	IN ONE LINE
D1 · Cleaning + attribution	<code>pipeline/load.py</code> , <code>prepare.py</code> , <code>attribute.py</code>	Files identified by column signature, typed, joined with row-count assertions; 140 untraced returns attributed by a validated model.
D2 · Scorecard	Web app → Overview; <code>scorecard.csv</code> written by the pipeline	Four dimensions, evidence-weighted, 0–100, per supplier and per supplier × material.
D3 · ₹ impact	Web app → each supplier; <code>money.py</code>	Problem-statement formula verbatim, then clearly separated additional losses.
D4 · Briefs	<code>web/public/briefs/*.pdf</code>	Five two-page A4 briefs: numbered asks, arithmetic, evidence POs, expected pushback.
D5 · Replace 3	Web app → Overview	VS03, VS19, VS12, with the best-scoring existing alternatives per material.

2 · What the data actually contains

Before modelling, a pre-flight script (`python -m pipeline.preflight`) tested every assumption the arithmetic depends on. The findings that changed the design:

- **The answer key is in the data.** `supplier_master.is_underperformer` flags VS03, VS12, VS19. It is removed at load time and kept sealed; no scoring or modelling code can reach it. It is opened once, after the ranking is final, to check the result (section 6).
- **Billing is on ordered quantity** on 100% of rows, so `invoice_amount_billed - quantity_received × unit_price_quoted` equals the value of material billed but not delivered. The rubric's formula and short-delivery loss are the same number.
- **The market index cannot judge individual suppliers.** It covers 6 of 18 materials, and where it exists the whole panel pays well above it (SS Pipes is on a different scale entirely). Price premium is measured against what other suppliers on the panel charged for the same material in the same quarter; the index is quoted only as context.
- **Unit prices are noisy** — every material spans roughly ₹45,000–₹1,25,000/MT — so no supplier-level premium survives a significance test (section 4).
- **The PS's suggested attribution rule does not fit the labelled data.** On returns whose supplier is recorded, that supplier is rarely the one whose batch of that material arrived most recently. Section 5 replaces it with a model and shows the difference.

- Smaller issues, handled and logged: `days_late` in payments does not reconcile with its own dates (recomputed); the year in `po_id` disagrees with `po_date` on most rows (time grouping uses `po_date`); 140 of 380 returns (37%) are untraced, close to the stated 35%.

3 • Financial quantification (40%)

Per purchase order, in rupees, then summed. Two tiers so a reviewer checking the stated formula finds exactly that number:

A • Short delivery = $\Sigma (\text{invoice_amount_billed} - \text{quantity_received} \times \text{unit_price_quoted})$

B • Customer returns = $\Sigma \text{quantity_returned} \times \text{the supplier's own median price for that material} \times P(\text{supplier})$

P = 1 for a recorded return; the model's probability for an untraced one. Each returned tonne is allocated exactly once.

Headline (the rubric's figure) = A + B. Additional, shown separately: material rejected at our inspection (rejected MT × quoted price), return handling (15% of return value — an assumption), and price premium only if statistically established.

SUPPLIER	RANK	A • SHORT	B • RETURNS	A + B	REJECTED	HANDLING	TOTAL
Tata Steel Service VS03	34	₹1,04,34,509	₹1,37,02,499	₹2,41,37,008	₹73,60,582	₹20,55,375	₹3,35,52,964
Thakur Steel VS19	33	₹1,05,89,582	₹1,36,93,152	₹2,42,82,734	₹74,08,247	₹20,53,973	₹3,37,44,954
Nutan Tubes VS12	32	₹1,19,34,606	₹1,49,43,845	₹2,68,78,451	₹82,28,767	₹22,41,577	₹3,73,48,795
Modern Metals VS17	31	₹13,79,869	₹29,21,067	₹43,00,936	₹0	₹4,38,160	₹47,39,096
Garg Enterprises VS16	30	₹14,23,221	₹22,88,190	₹37,11,411	₹0	₹3,43,228	₹40,54,639
Sai Steel VS11	29	₹13,32,436	₹23,80,491	₹37,12,927	₹0	₹3,57,074	₹40,70,001
All 34 suppliers				₹15,98,87,902	₹2,29,97,595	₹1,21,82,591	₹19,50,68,088

Sanity check against the problem statement. The PS estimates ₹8–12L a year lost to the three underperformers on ₹70–85L monthly procurement. The supplied data transacts ₹27.26 Cr a month, so absolute figures are larger. Measured as a rate — what the bottom three leak *beyond* the rest of the panel's rate on the same spend — they lose 0.95% of spend. Applied to the stated procurement that is **₹7.95 L – ₹9.65 L a year**, inside the PS's own range, reached independently.

4 • Scorecard (D2)

DIMENSION	WEIGHT	MEASURE
Short delivery	30%	Quantity not delivered ÷ quantity ordered
Quality rejection	25%	(Rejected at our inspection + customer returns attributed) ÷ quantity received
Late delivery	25%	Average days late per order. Nearly every supplier is a day or two late on most orders, so “% late” barely separates them; it is still reported.
Price premium	20%	Average premium over other suppliers, same material and quarter

- **Scored against the typical supplier, not the worst.** Each dimension scores 100 at or better than the panel median, falling to 0 at 6 robust standard deviations worse, where a unit is never smaller than a commercially material difference (0.25 pp short, 0.5 days late, 0.25 pp quality, 1.0 pp price). Scaling to the panel's best and worst would guarantee someone scores 0 even when nobody is bad; this does not (see section 9, “Tested on data with a known answer”).

- **Fair to small suppliers.** Each rate is shrunk toward the panel average by $(n \cdot \text{rate} + k \cdot \text{panel}) \div (n + k)$, $k = 5$ orders, so a handful of bad orders cannot top the worst list. Price premium is shrunk by its own noise (James–Stein): the spread between suppliers is no larger than order-to-order noise would produce, so the dimension carries little weight in practice — which is the honest outcome.
- **Price premium is never charged without evidence.** One-sided t-test per supplier, Benjamini–Hochberg correction across all 34. None survives at 5% FDR, so ₹0 is claimed. The briefs still ask for price alignment on materials where the supplier’s own median sits above the panel’s — a request backed by their orders, not a loss claim.
- **Per material, not just per supplier.** Every metric is also computed per supplier × material (shrunk toward that supplier’s own rate) and flagged where it is at least 2× the panel’s rate for that material. These drive the brief’s material-specific asks and the D5 alternatives.
- **Relationship length is not an input.** No scoring code reads `years_of_relationship`; a test shuffles it and asserts every score is unchanged.
- **Robust to the weights.** The bottom three are identical under all 5 weightings tested (base, equal, money-heavy, service-heavy, price-heavy).

5 • Missing-attribution inference (35%)

A return records material, date, quantity and reason — but no PO or batch — so it cannot be joined to a delivery. It is treated as a ranking problem: *of all suppliers who had delivered before this return, how likely is each to be its source?*

1. **Candidates.** Every supplier with a delivery on or before the return date.
2. **Features**, each computed only from deliveries dated on or before the return (a test adds catastrophic future deliveries and asserts no feature changes):
 - How recently they delivered this material (weight +0.04)
 - How much of this material they supplied recently (weight -0.18)
 - How much of their material we rejected (weight +0.29)
 - How often they delivered short (weight -0.28)
 - How late they usually deliver (weight -0.05)
 - How many returns were already traced to them (weight +0.99)

The return-history feature uses labels, so under cross-validation it is rebuilt inside each fold from the training returns only.

3. **Model.** Logistic regression on (return, supplier) pairs, probabilities normalised per return, then blended with plain exposure (each supplier’s recent share of that material) at the weight held-out log loss prefers — 1.00 on the model for this data. It therefore cannot do worse than that simple rule, and falls back to it when the data has no learnable signal. 7 parameters; fits in under a second; refits automatically on any new dataset.
4. **Validation.** 5-fold cross-validation grouped by return, on the 240 returns whose supplier is recorded: hide the answer, predict, compare.

METHOD	VALUE MISALLOCATED ↓	RIGHT SUPPLIER IN TOP 3 ↑	RANKED FIRST ↑
Our model	5.3%	55.8%	20.8%
Blame whoever delivered that material last (the assignment’s rule)	47.9%	10.2%	4.0%
Split by who supplied most of that material	47.1%	6.7%	0.8%
Split equally between all suppliers	47.0%	8.8%	2.9%

Why misallocation is the headline metric. The rupee totals need each supplier’s *share* of return value to be right, not every individual return. Misallocation measures exactly that: the fraction of held-out return value that lands on the wrong supplier once summed per supplier. The model gets 95% right; every rule-based alternative, including the PS’s, gets about half.

Honest limits. Top-1 accuracy is capped by the data: several suppliers could have supplied any given return, and nothing on the return distinguishes them. The model’s confidence reflects this and is calibrated (10%-20%: predicted 19%, observed 22%; 20%-30%: predicted 22%, observed 20%). That is why inferred returns are split by probability in the totals, shown as an estimate in the briefs, and **never demanded as a credit** — only recorded returns are.

With fewer than 10 labelled returns the pipeline does not train; it falls back to the most-recent-batch rule and says so in every output (tested).

6 • Blind validation

With the ranking frozen, the sealed `is_underperformer` column was opened for the first time. It flags VS03, VS12, VS19. Our bottom three, computed without it: **VS03, VS19, VS12** — ranks VS03 #34, VS12 #32, VS19 #33 of 34, 69 score points clear of the next supplier. The column was a check on the method, never an input to it.

7 • Negotiation briefs (25%) and recommendation

Briefs are generated for the bottom 5 by score. Each is two A4 pages: page one carries the rupee figure with its arithmetic, three or four numbered asks with the evidence for each, likely counter-arguments, and a walk-away position; page two lists the purchase orders to put on the table and a per-material breakdown. Rules: every ask cites a figure the supplier can check against their own invoice or our GRN; only recorded returns are claimed; the price ask quotes the panel median for the material. Example (VS03): *“Issue a credit note of ₹1,04,34,509 for material billed but not delivered, and invoice on GRN-received quantity from the next order.”*

REPLACE	COST TO US	AVOIDABLE BY SWITCHING	LARGEST MATERIALS → BEST EXISTING ALTERNATIVES
Tata Steel Service VS03	₹3.36 Cr	₹3.01 Cr	MS Flats → Ratan Steel, Prakash Industries; MS Rods → MSP Steel, Laxmi Hardware; SS Pipes → Viraj Profiles, Shyam Metalics
Thakur Steel VS19	₹3.37 Cr	₹3.04 Cr	CR Sheets → Global Metals, Bhushan Steel; GI Wire → Precision Steel, Prakash Industries; ERW Pipes → Bhavani Metals, Om Metals
Nutan Tubes VS12	₹3.73 Cr	₹3.36 Cr	HR Coils → Viraj Profiles, Western Steel; Spring Steel → SAIL Distribution, Bhushan Steel; MS Rods → MSP Steel, Laxmi Hardware

8 • Assumptions and limitations

- Return value uses the supplier’s own median price for the material returned; handling cost (15%) is an assumption and sits outside the headline.
- The late-delivery penalty in the briefs (0.5%/day, capped at 5%) is a negotiating position; it is never added to any loss total.
- Arora’s own late payment (recomputed from invoice date + agreed days) is surfaced in each brief as the supplier’s likely counter-argument, not scored.
- Attribution assumes the labelled returns are representative of the unlabelled ones; nothing in the data suggests otherwise, but it cannot be proven.
- All results are refit from scratch on any new data; there is no live retraining on the web app by design — it serves precomputed results.

9 • Engineering

One command. `python -m pipeline.run` reads the six CSVs and writes the scorecard, the attribution, all JSON

Tested on data with a known answer. `python -m evaluation.run` generates nine datasets in this schema with planted offenders and the true source of every

for the web app, CSV exports, five PDF briefs and this document, in about 19 seconds (most of it printing PDFs).

Nothing tuned to this dataset. Files are found by their columns, not names; suppliers, materials and thresholds are discovered; every constant lives in `config.py` with its reasoning. A test runs the full pipeline on a synthetic dataset with different IDs, materials, file names and size, and checks it finds that dataset's bad suppliers.

return — including a 120-supplier panel, suppliers that deteriorate only in the final year, and files as Excel exports them in India (DD/MM/YYYY dates, 12,34,567.89 amounts) — and scores the unchanged pipeline: all planted offenders found in each, the right failure dimension named, rupee figures exact, attribution better than the problem statement's rule on hidden returns, and no false alarms on a panel where nobody is bad (`evaluation/REPORT.md`).

Fails loudly. Missing files, mixed units and fanned-out joins stop the run with a specific message. Everything else the pipeline notices (23 findings on this data) is logged with the action taken and shown on the Method page.

Stack. Python (pandas, scikit-learn, SciPy, Jinja2) for analysis; React + TypeScript + Tailwind, built with Vite, for a static web app that does no computation and cannot go down. An automated test suite guards each claim made here.